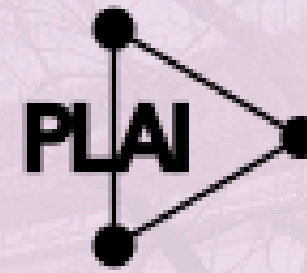


LMU

LUDWIG-
MAXIMILIANS-
UNIVERSITÄT
MÜNCHEN

FACULTY OF MATHEMATICS, INFORMATICS AND STATISTICS
INSTITUTE OF INFORMATICS

**CHAIR OF PROGRAMMING LANGUAGES
AND ARTIFICIAL INTELLIGENCE**



Binary Analysis with AI

Project Presentation

Presented by:

Montassar Arfaoui

Supervisor: Yunru Wang

Chair Professor: Prof. Dr. Johannes Kinder

Date of the presentation:
13th February 2026

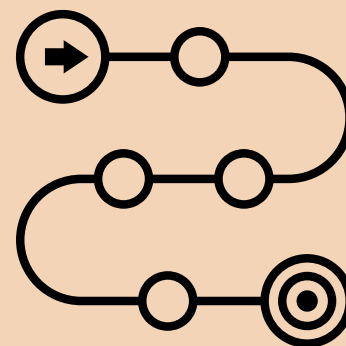


Understanding the Project: Key Questions

**What is PalmTree?
Why did I conduct this study?**



**What did I do?
& Challenges?**



What did we learn?



Slide 3

Slide 8

Slide 16

Why did I conduct this project?

- Overview and Introduction to PalmTree
- Limitations and Challenges of the PalmTree Model
- Research Objectives and Key Contributions

PALMTREE: Learning an Assembly Language Model for Instruction Embedding

Xuezixiang Li

University of California Riverside
Riverside, CA 92521, USA
xli287@ucr.edu

Yu Qu

University of California Riverside
Riverside, CA 92521, USA
yuq@ucr.edu

Heng Yin

University of California Riverside
Riverside, CA 92521, USA
heng@cs.ucr.edu

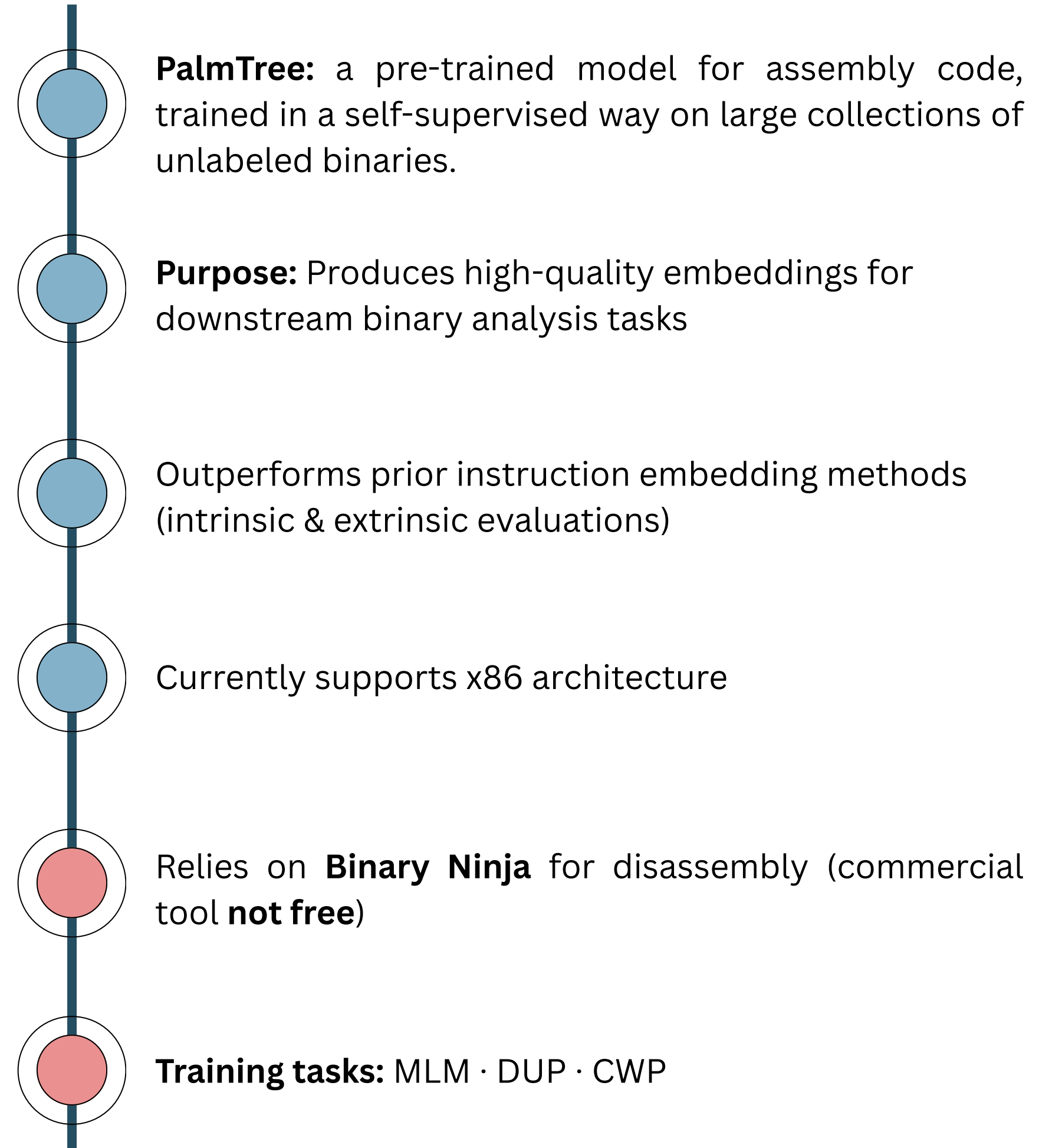
What is PalmTree?



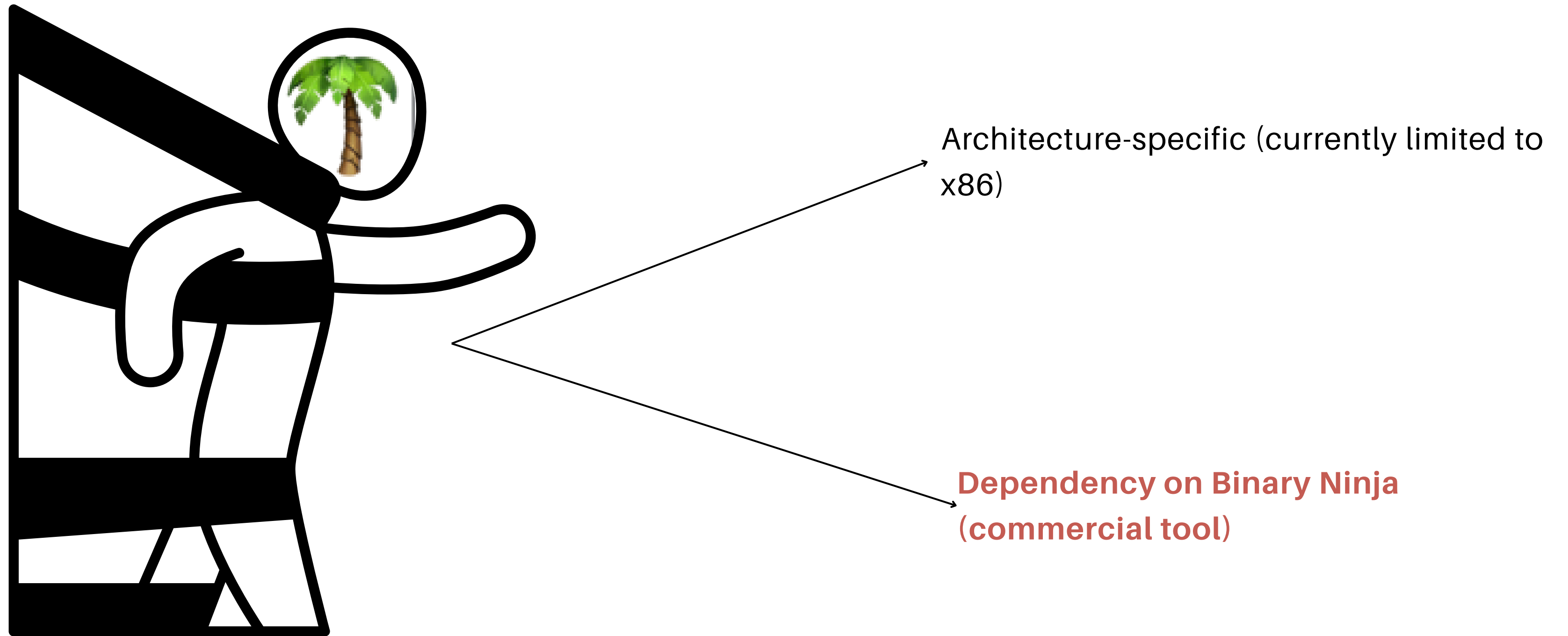
Official repository for 🌴

<https://github.com/palmtreemodel/PalmTree>

Official paper: <https://github.com/palmtreemodel/PalmTree>

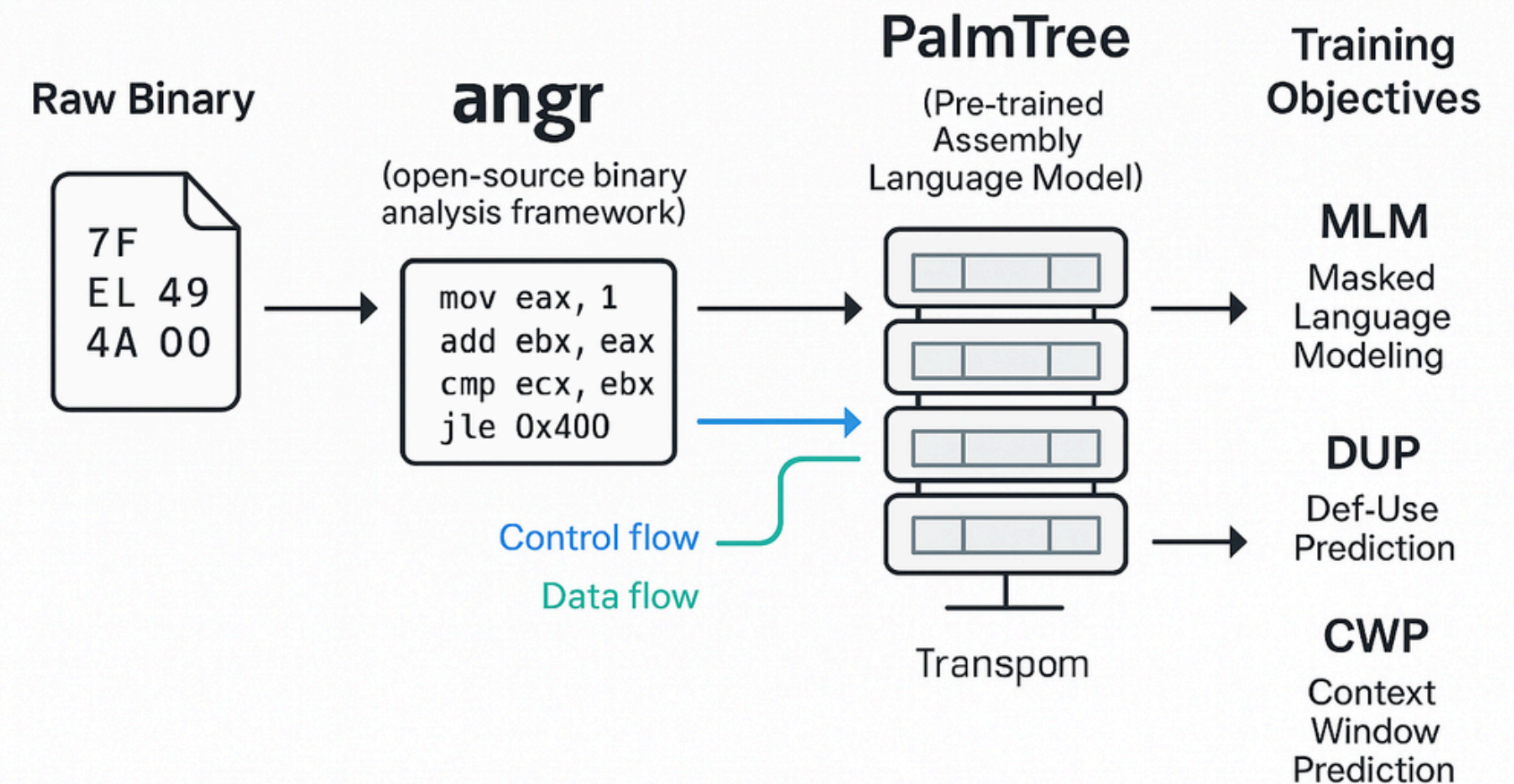


These points are the most relevant for this study.



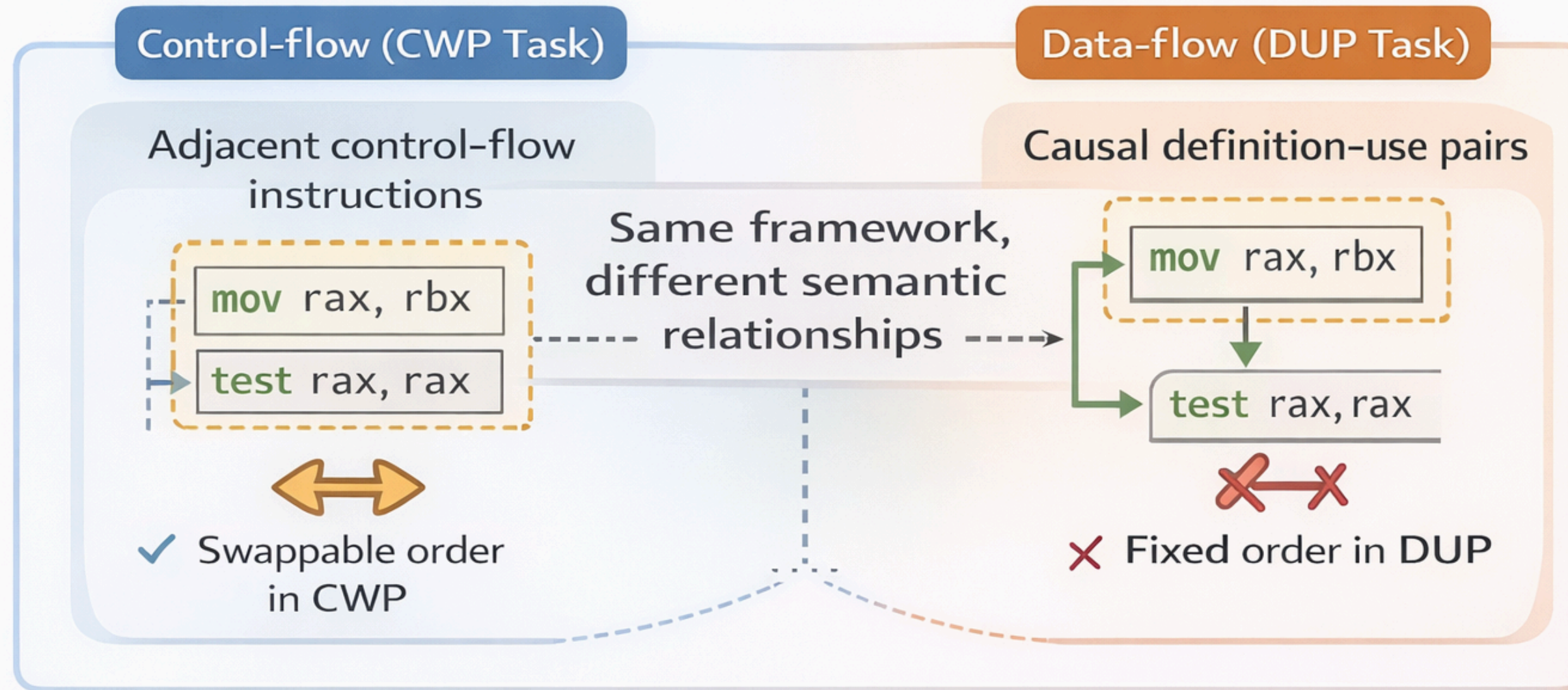
Fisrt Primary Objective

- **Objective:** Improve PalmTree's reusability by eliminating dependencies on commercial tools and enabling a fully free pipeline.



CWP + DUP + MLM

PALMTREE uses multiple tasks to learn control-flow and data-flow semantics



⚠ **Mismatch** possible: valid, swappable CWP pairs might violate DUP order!

Second Primary Objective

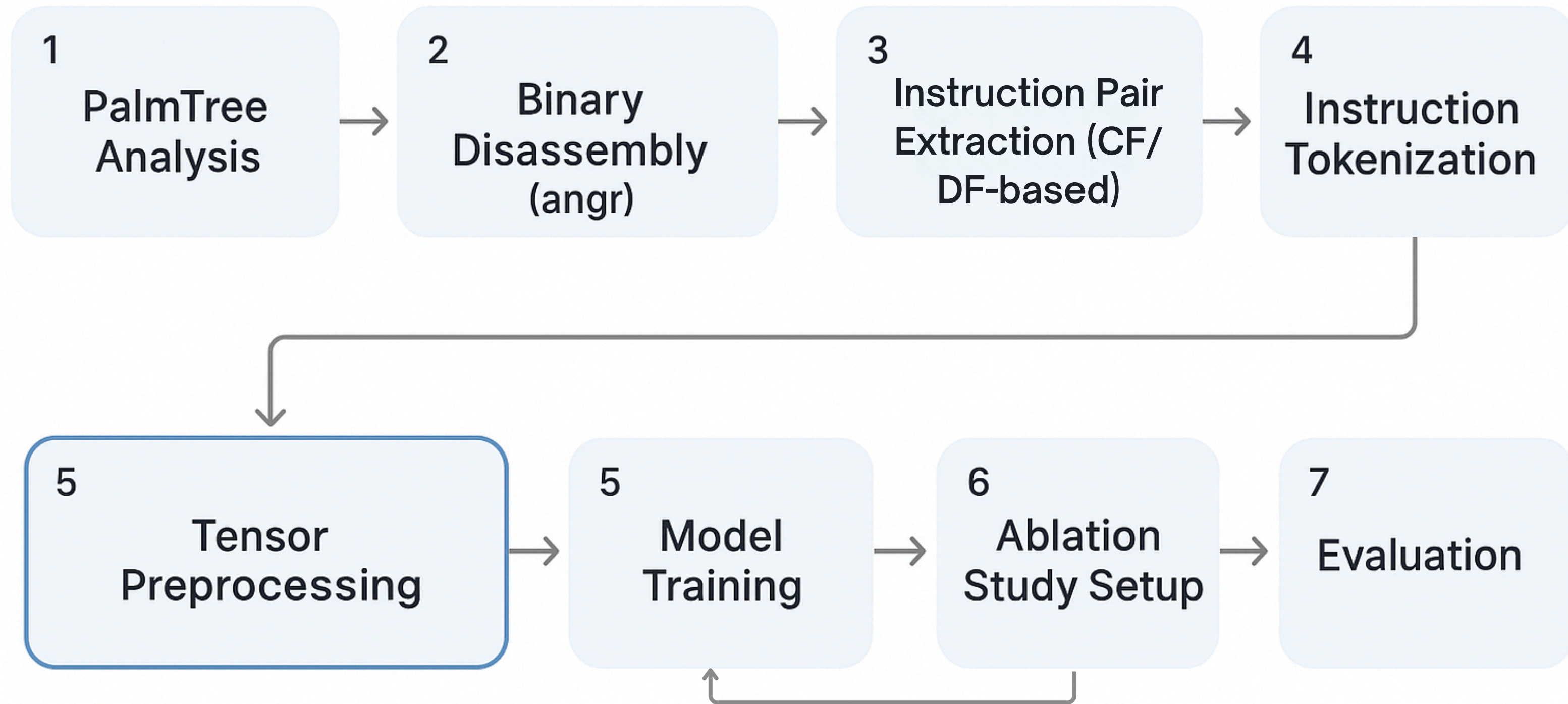
- **Research Question:** What is the contribution of each task to the model's overall performance?
- **Objective:** Perform an ablation study on PalmTree's training tasks

Model Variant	MLM	CWP	DUP
PalmTree-M	X	-	-
PalmTree-MD	X	-	X
PalmTree-MC	X	X	-
Full PalmTree	X	X	X

What did I do?

- Overview of the key implementation steps
- Main challenges encountered during the project

Overview of Performed Steps



Environment setup

- Environment Setup
 - Conda Environments:
 - palmtree-train: model training and evaluation
 - angr: binary analysis and input generation
- Dataset Split
 - Training set: 5 projects
 - Evaluation set: 2 projects
 - Ensured strict separation between training and evaluation data



Dataset-1.zip



NumPy

Binary Disassembly (angr)

CFG & DFG xExtraction

Instruction Tokenization

- Generated PalmTree inputs with angr
- Derived instruction pairs from CFG/DFG edges
- Tokenized instructions (fine-grained, PalmTree style)
- Output: tokenized instruction sequences

CFG-based instruction pairs

```
GNU nano 6.2
ub rsp, 8      mov rax, qword ptr [rip + 0x497f85]
ov rax, qword ptr [rip + 0x497f85]      test rax, rax
est rax, rax   je 0x48a00a
e 0x48a00a     call 0x58cb96
all 0x58cb96   call rax
all rax       add rsp, 8
dd rsp, 8     ret
ush qword ptr [rip + 0x497fe2] jmp qword ptr [rip + 0x497fe4]
op dword ptr [rax]      jmp qword ptr [rip + 0x497fe2]
ea rax, [rip + 0x499290] lea rdi, [rip + 0x499282]
ea rdi, [rip + 0x499282] push rbp
ush rbp       sub rax, rdi
ub rax, rdi   mov rbp, rsp
ov rbp, rsp   cmp rax, 0xe
mp rax, 0xe   ja 0x48b01d
a 0x48b01d    pop rbp
op rbp ret
et          mov rax, qword ptr [rip + 0x496f94]
ov rax, qword ptr [rip + 0x496f94]      test rax, rax
est rax, rax   je 0x48b01b
e 0x48b01b     pop rbp
op rbp jmp rax
op dword ptr [rax]      lea rax, [rip + 0x499259]
ea rax, [rip + 0x499259] lea rdi, [rip + 0x499252]
ea rdi, [rip + 0x499252] push rbp
```

DFG-based instruction pairs

```
GNU nano 6.2
all rax       add rsp 8
dd rsp 8     ret
dd rsp 8     ret
ub rsp 8     mov rax qword ptr [rip + address]
ov rax qword ptr [rip + address]      test rax rax
est rax rax   je address
e address     call rax
all rax       add rsp 8
dd rsp 8     ret
ov rax qword ptr [rip + address]      test rax rax
est rax rax   je address
e address     call rax
all rax       add rsp 8
dd rsp 8     ret
est rax rax   je address
e address     add rsp 8
dd rsp 8     ret
e address     call rax
all rax       add rsp 8
dd rsp 8     ret
ush qword ptr [rip + address] jmp qword ptr [rip + address]
or ebp ebp   mov r9 rdx
ov r9 rdx     pop rsi
op rsi mov rdx rsp
ov rdx rsp    and rsp address
nd rsp address push rax
```

Tensor Preprocessing

- Preprocessed data offline following the PalmTree pipeline
- Persisted preprocessed tensors to disk
- Improved training stability and efficiency

```
print("Creating Dataloader")
train_data_loader = DataLoader(train_dataset, batch_size=256, num_workers=10)
```

```
test_data_loader = DataLoader(test_dataset, batch_size=256, num_workers=10) \
    if test_dataset is not None else None
```

```
"""
Here we implement a class for loading data.
"""
```

```
import torch
from torch.autograd import Variable
from vocab import *
from config import *
import numpy as np
import random
import re
```

```
np.random.seed(0)
```

```
class DataLoader:
    EOS = 0 # to mean end of sentence
    UNK = 1 # to mean unknown token
    |
```

Ablation Study Setup & Training Variants

- Identified how individual training tasks are implemented and can be disabled.
- Configured training variants and trained them for comparison:
 - Full PalmTree (MLM + CWP + DUP)
 - MLM only
 - MLM + CWP
 - MLM + DUP

```
# Setting the tqdm progress bar
data_iter = tqdm.tqdm(enumerate(data_loader),
                      desc="EP_%s:%d" % (str_code, epoch),
                      total=len(data_loader),
                      bar_format="{l_bar}{r_bar}")

avg_loss = 0.0
total_correct = 0
total_element = 0

for i, data in data_iter:
    # 0: batch data will be sent into the device(GPU or CPU)
    data = {key: value.to(self.device) for key, value in data.items()}

    # 1. forward the next_sentence_prediction and masked_lm model
    dfg_next_sent_output, cfg_next_sent_output, mask_lm_output = self.model.forward(data["dfg_bert_input"], data["dfg_segment_label"], data["cfg_bert_input"], data["cfg_segment_label"])

    # 2-1. NLL(negative log likelihood) loss of is_next classification result
    dfg_next_loss = self.dfg_next_criterion(dfg_next_sent_output, data["dfg_is_next"])
    cfg_next_loss = self.cfg_next_criterion(cfg_next_sent_output, data["cfg_is_next"])

    # 2-2. NLLloss of predicting masked token word
    mask_loss = self.masked_criterion(mask_lm_output.transpose(1, 2), data["dfg_bert_label"])

    # 2-3 NLLloss of instruction component prediction
    comp_loss = self.comp_criterion(inst_comp_output.transpose(1, 2), data["component"])

    # 2-5. Adding next_loss and mask_loss : 3.4 Pre-training Procedure
    loss = dfg_next_loss + cfg_next_loss + mask_loss

    # 3. backward and optimization only in train
    if train:
        self.optim_schedule.zero_grad()
        loss.backward()
        self.optim_schedule.step_and_update_lr()

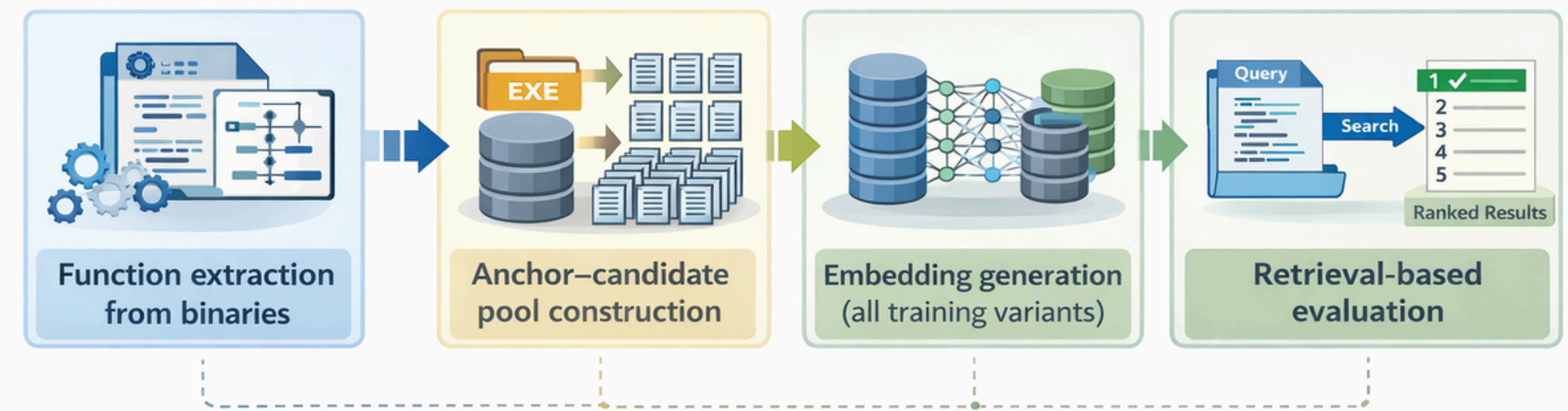
    post_fix = {
        "epoch": epoch,
        "iter": i,
        "CWP": cfg_next_loss.item(),
        "DUP": dfg_next_loss.item(),
        "MLM": mask_loss.item(),
    }
```

Evaluation Comparison

Evaluation Setup (Key Idea)

- Positives: same function, different compiler settings
- Negatives: different functions
- Evaluation on ~28k anchors

Evaluation Pipeline

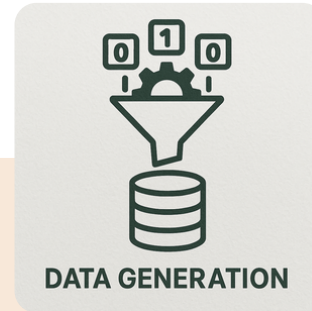


Metrics:

- Recall@1 • Recall@10 • Mean Reciprocal Rank

```
(base) ru94wid@aranuka:~/projects/PalmTree/evaluation$ wc -l functions.jsonl
44760 functions.jsonl
(base) ru94wid@aranuka:~/projects/PalmTree/evaluation$
```

Challenges



During input generation

- Tool-specific differences in binary analysis
 - Mid-level IR availability
 - Implicit dependency handling (e.g., EFLAGS updates)
 - CFG stability and normalization
- Ensuring compatibility with PalmTree's original data extraction pipeline
- Handling CFG and DFG consistency across binaries



During training and evaluation

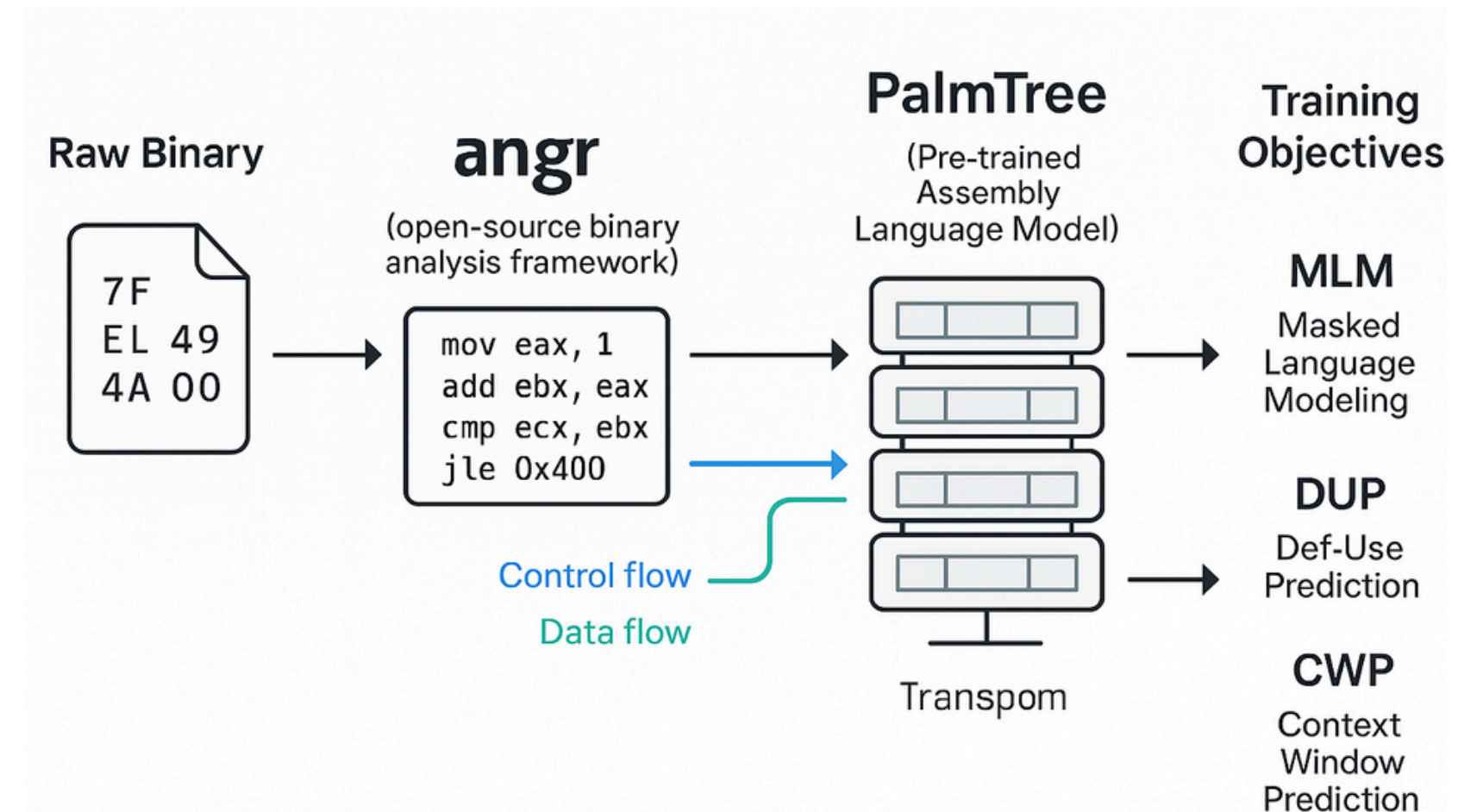
- Parallel preprocessing during training
 - Caused long training times and out-of-memory (OOM) errors on LMU servers and hardware.
 - High computational cost and long training times
- Managing multiple training variants for ablation studies
- Ensuring fair and consistent evaluation across models

What did this project achieve?

- Results, Findings & Analysis

Objective: Make PalmTree Fully Free

- Reliable performance with a fully open-source pipeline
 - angr + PalmTree yields strong, stable metrics
 - Open-source implementation is effective and efficient

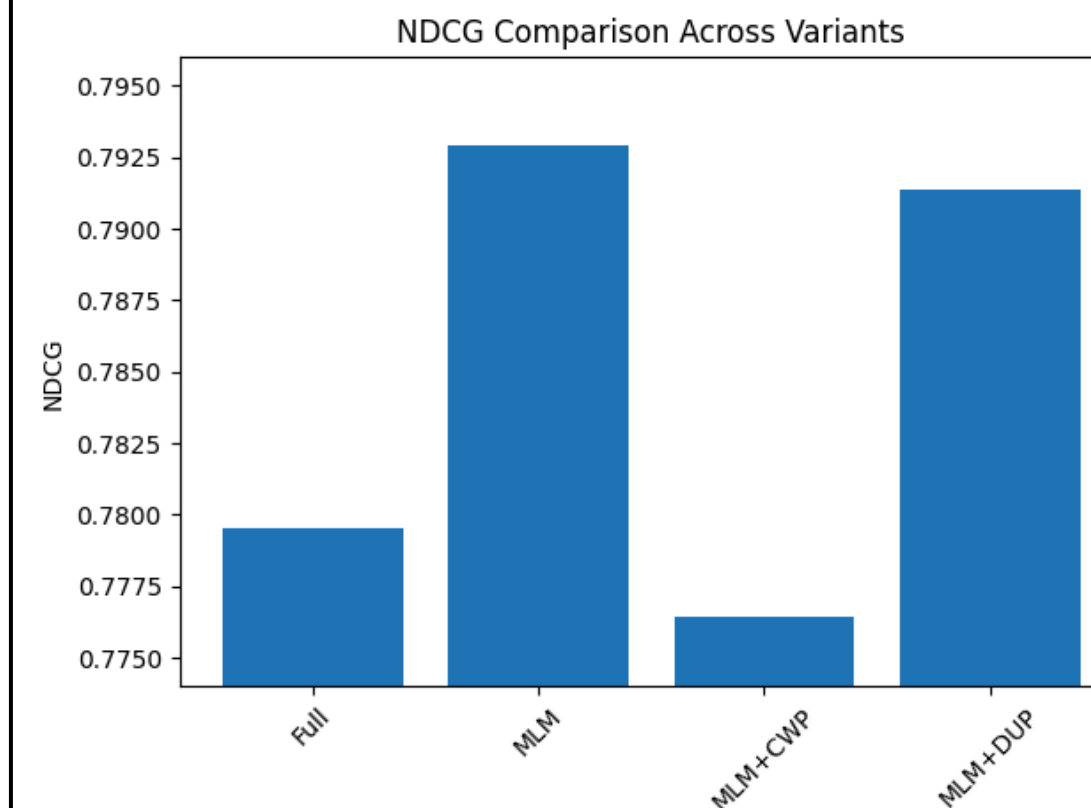
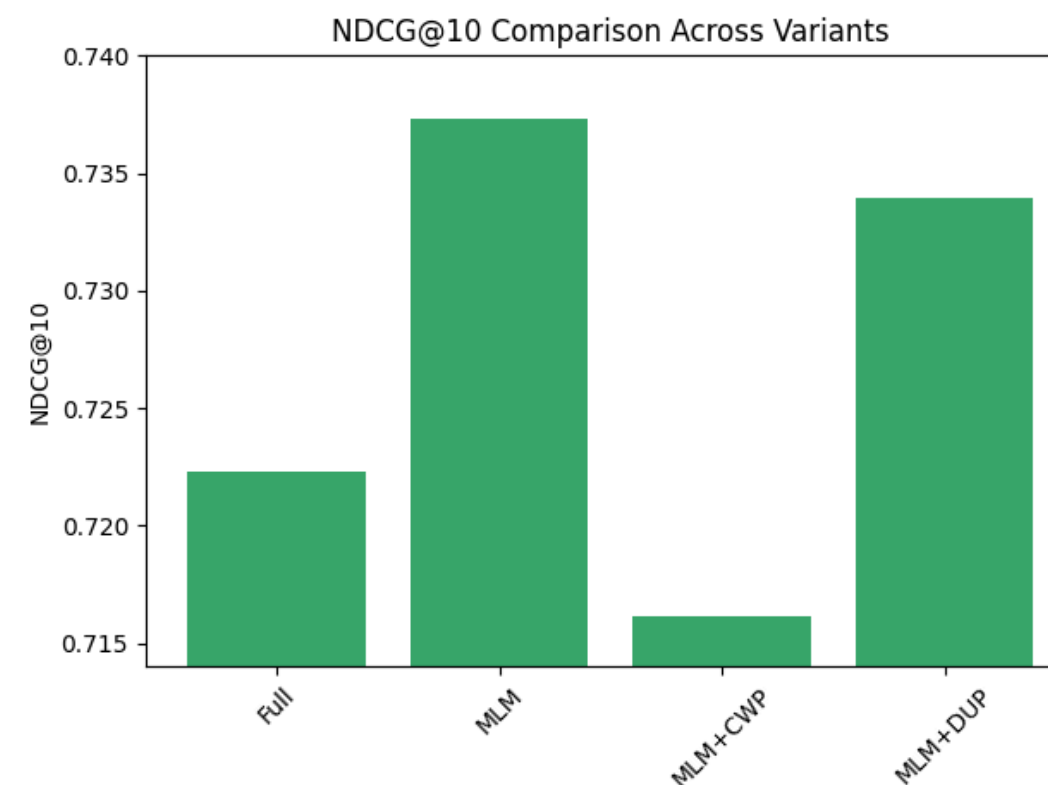
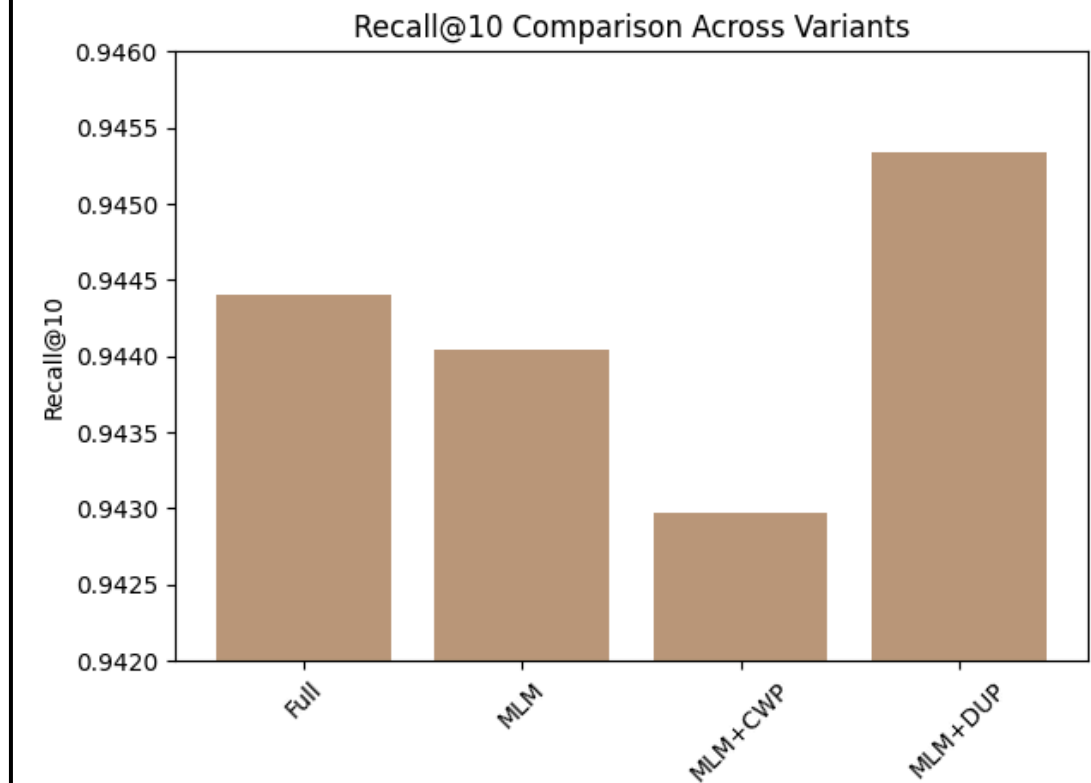
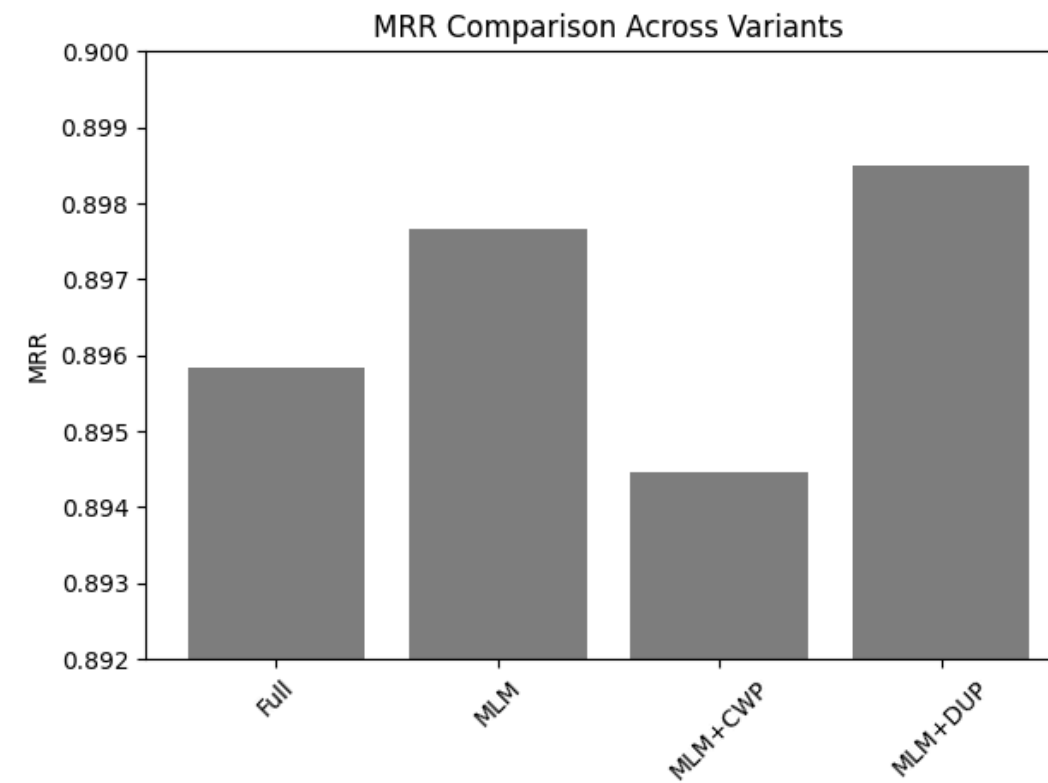


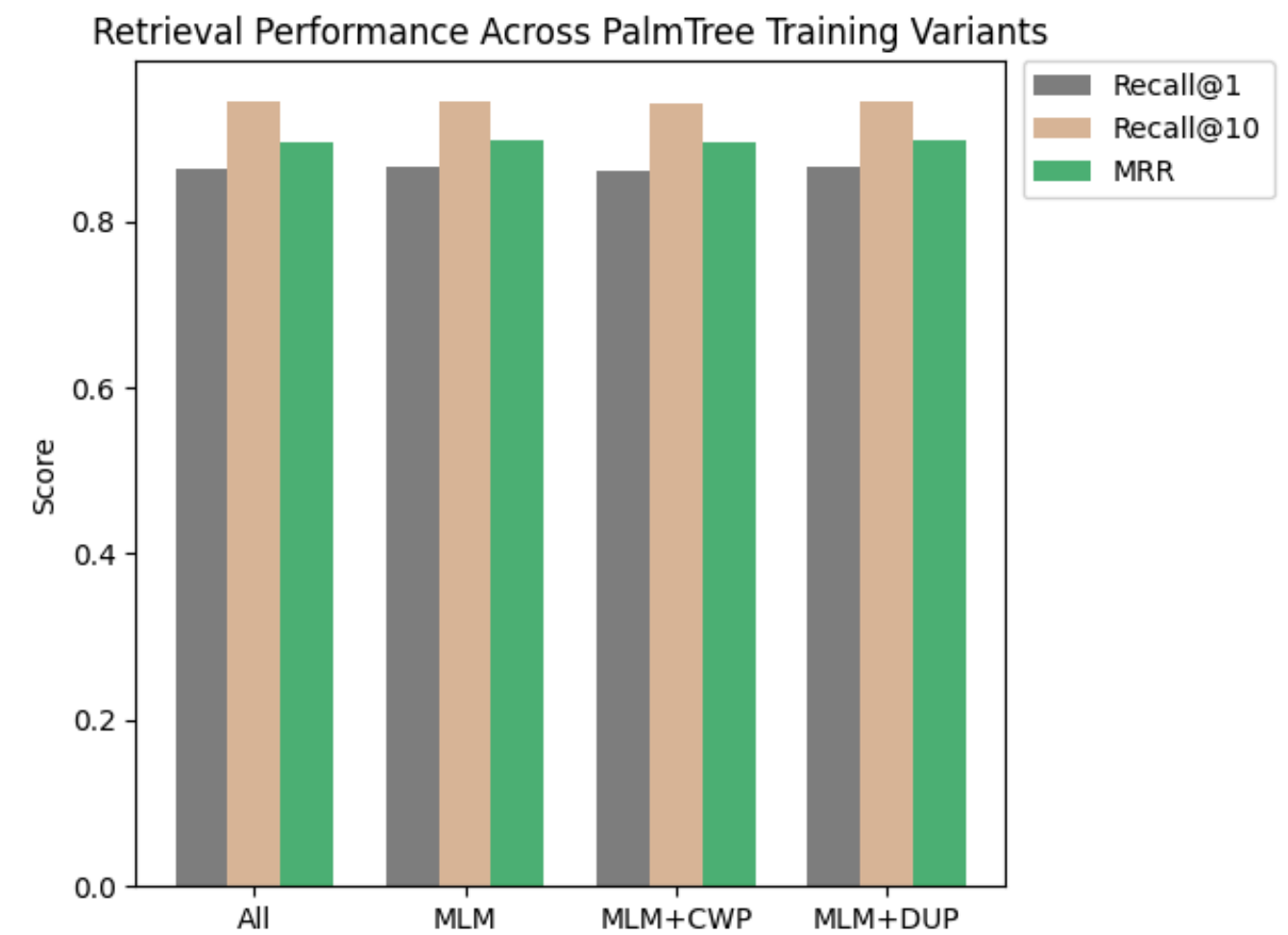
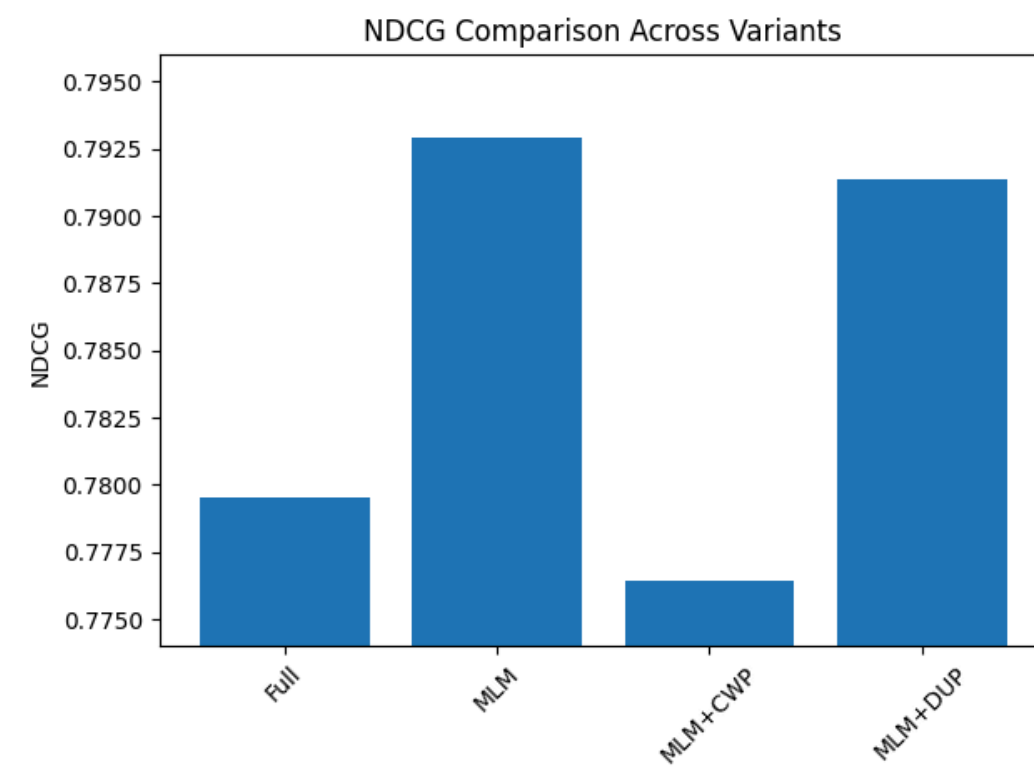
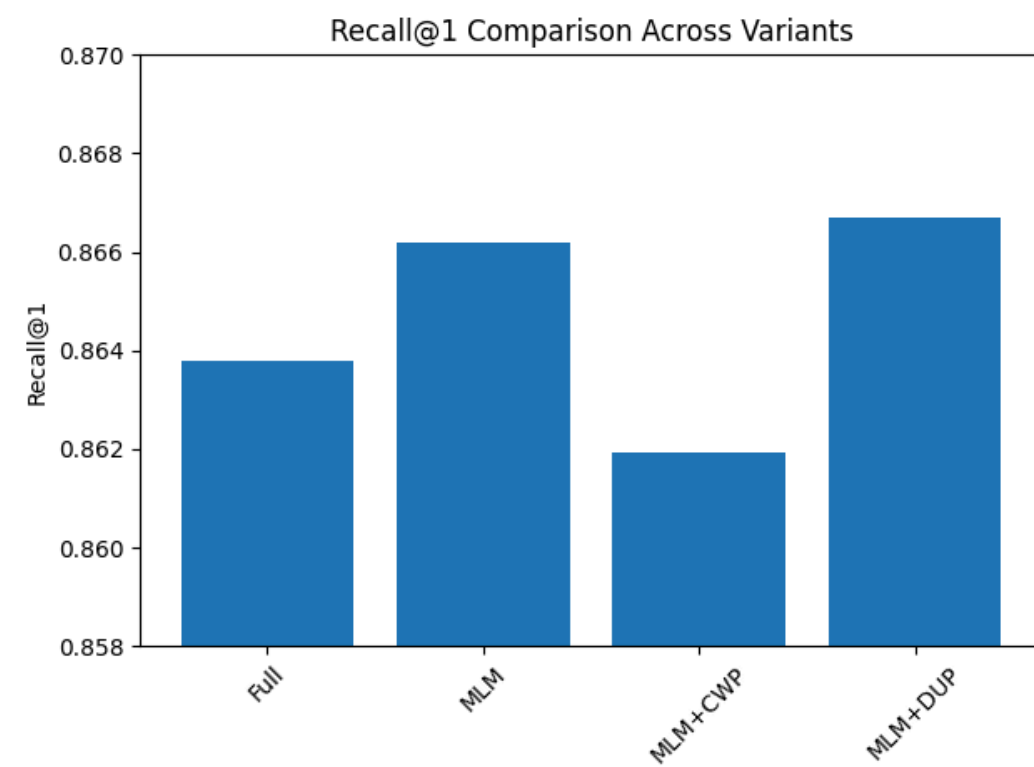
Results

```
[{
  [
    "model": "embeddings_FullTraining_All.npy",
    "num_anchors_evaluated": 27845,
    "Recall@1": 0.8637816484108457,
    "Recall@10": 0.9444065361824385,
    "MRR": 0.8958475017384363,
    "NDCG": 0.7794988984522181,
    "NDCG@10": 0.7222652008951365
  ]
}]
(palmtree-train) ru94wid@aranuka:~/projects/PalmTree$
```

Ablation study results

- All PalmTree variants achieve strong retrieval performance
- Results are very similar across configurations (robustness)
- **Recall@10:** Percentage of anchor functions for which at least one correct (similar) function appears within the top 10 ranked candidates (based on cosine similarity).





PALMTREE: Learning an Assembly Language Model for Instruction Embedding

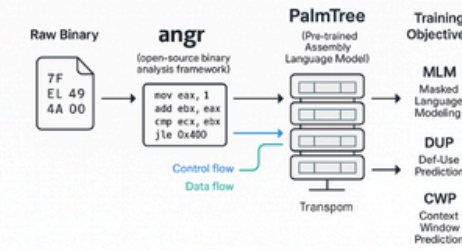
Xuezixiang Li
University of California Riverside
Riverside, CA 92521, USA
xli287@ucr.edu

Yu Qu
University of California Riverside
Riverside, CA 92521, USA
yuq@ucr.edu

Heng Yin
University of California Riverside
Riverside, CA 92521, USA
heng@cs.ucr.edu

Fisrt Primary Objective

- **Objective:** Improve PalmTree's reusability by eliminating dependencies on commercial tools and enabling a fully free pipeline.



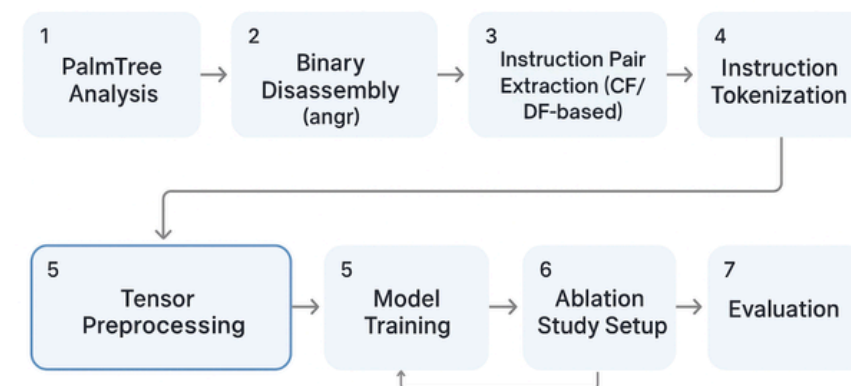
Second Research Question and Objective

- **Research Question:** Is using all three training tasks (MLM, DUP, CWP) necessary for optimal performance?

- **Objective:** Perform an ablation study on PalmTree's training tasks

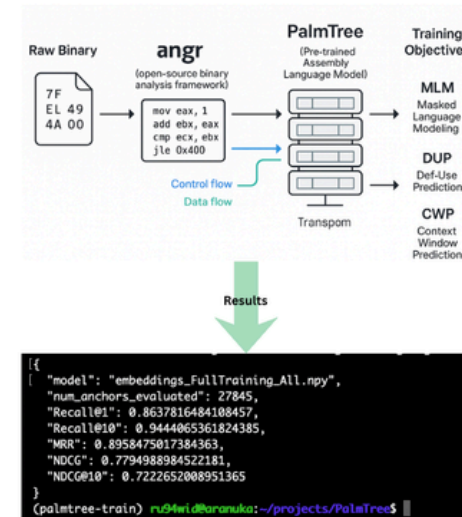
Model Variant	MLM	CWP	DUP
PalmTree-M	X	-	-
PalmTree-MD	X	-	X
PalmTree-MC	X	X	-
Full PalmTree	X	X	X

Overview of Performed Steps



Objective: Make PalmTree Fully Free

- Reliable performance with a fully open-source pipeline
 - angr + PalmTree yields strong, stable metrics
 - Open-source implementation is effective and efficient



Ablation study results

- All PalmTree variants achieve strong retrieval performance
- Results are very similar across configurations (robustness)
- **Recall@10:** Percentage of anchor functions for which at least one correct (similar) function appears within the top 10 ranked candidates (based on cosine similarity).

